

Monotonic Stack & Next Greater Element

Data Structures & Algorithms Guide

September 19, 2026

1 What is a Monotonic Stack?

A **monotonic stack** is a stack where elements are always kept in a specific order — either strictly increasing or strictly decreasing from bottom to top. It is not a different data structure, but rather a standard stack used with a specific discipline: before pushing a new element, you pop off anything that violates the desired monotonic order.

1.1 Monotonic Decreasing Stack

In a monotonic decreasing stack, elements go from largest at the bottom to smallest at the top.

- **Operation:** Before pushing an element x , pop everything $\leq x$.
- **Use Case:** Used for solving **Next Greater Element** problems.

```
1 def monotonic_decreasing_stack(nums):
2     stack = []
3     for num in nums:
4         # Maintain decreasing order: pop elements <= current num
5         while stack and stack[-1] <= num:
6             stack.pop()
7         stack.append(num)
8     return stack
```

Listing 1: Monotonic Decreasing Stack Template

1.2 Monotonic Increasing Stack

In a monotonic increasing stack, elements go from smallest at the bottom to largest at the top.

- **Operation:** Before pushing an element x , pop everything $\geq x$.
- **Use Case:** Used for solving **Next Smaller Element** problems.

```
1 def monotonic_increasing_stack(nums):
2     stack = []
3     for num in nums:
4         # Maintain increasing order: pop elements >= current num
5         while stack and stack[-1] >= num:
6             stack.pop()
```

```

7     stack.append(num)
8     return stack

```

Listing 2: Monotonic Increasing Stack Template

2 Why It Works --- Intuition

Think of the stack as candidates waiting for their answer. As you scan through the array, you keep a stack of numbers that haven't found a bigger number yet.

The moment you encounter a bigger number, it serves as the answer for whatever is on top of the stack — you pop the element and record the answer. Anything left in the stack at the end of the scan never found a bigger number, meaning no greater element exists for those positions.

3 Worked Example

Below is a step-by-step trace table demonstrating the Next Greater Element logic on the array $[2, 1, 2, 4, 3]$ using a monotonic decreasing stack.

Table 1: Trace table for Next Greater Element on array $[2, 1, 2, 4, 3]$

Step	num	Stack before	Action (pops)	Stack after
1	2	$[]$	—	$[2]$
2	1	$[2]$	no pop ($1 < 2$)	$[2, 1]$
3	2	$[2, 1]$	pop 1 ($1 \leq 2$)	$[2, 2]$
4	4	$[2, 2]$	pop 2, pop 2	$[4]$
5	3	$[4]$	no pop ($3 < 4$)	$[4, 3]$

4 Time and Space Complexity

- **Time Complexity: Amortized $\mathcal{O}(n)$**

Even though there is a nested `while` loop inside the `for` loop, each element in the array is pushed onto the stack exactly once and popped at most once across the entire algorithm execution. Therefore, the total work done is bounded by $\mathcal{O}(n)$ pushes + $\mathcal{O}(n)$ pops = $\mathcal{O}(n)$ time complexity, not $\mathcal{O}(n^2)$.

- **Space Complexity: $\mathcal{O}(n)$**

$\mathcal{O}(n)$ auxiliary space is required for the result array, plus $\mathcal{O}(n)$ space in the worst case for the stack (for instance, when given a strictly decreasing array where no element ever gets popped during iteration).

5 Python Implementation

Here is the complete Python function implementing the Next Greater Element algorithm using a right-to-left iteration with a monotonic stack:

```

1 def nge(nums):
2     n = len(nums)
3     res = [0] * n
4     stack = []
5     for i in range(n - 1, -1, -1):
6         while stack and stack[-1] <= nums[i]:
7             stack.pop()
8         if stack:
9             res[i] = stack[-1]    # top of stack = next greater element
10        else:
11            res[i] = -1           # nothing greater found
12            stack.append(nums[i]) # push current element for future
13                                # comparisons
14        return res
15
16 nums = [1, 3, 4, 2]
17 print(nge(nums))

```

6 Dry Run of the Code Above

Below is a step-by-step dry run of `nge([1, 3, 4, 2])` traversing the array from right to left (index $i = 3$ down to 0):

Table 2: Dry run of `nge([1, 3, 4, 2])`

Index i	nums[i]	Stack before	While loop pops	res[i] assigned	Stack after
3	2	[]	None	-1	[2]
2	4	[2]	pop 2 ($2 \leq 4$)	-1	[4]
1	3	[4]	None ($4 > 3$)	4	[4, 3]
0	1	[4, 3]	None ($3 > 1$)	3	[4, 3, 1]

Final Output: `res = [3, 4, -1, -1]`

7 Where This Pattern Is Used

The monotonic stack pattern is fundamental in algorithm design and frequently used in problems such as:

- Next Greater / Next Smaller Element
- Largest Rectangle in Histogram
- Daily Temperatures
- Trapping Rain Water
- Stock Span Problem